CURSO DE BIG DATA E IA · BLOQUE UPDATE

Inteligencia Artificial

Modelos de IA y aprendizaje automático, en dos sesiones de 4 horas.

Sesión 1 · Modelos de IA **Sesión 2 · Aprendizaje automático**

Contenidos basados en los módulos 5071 y 5072 (BOE núm. 111, 10 de mayo de 2021)

01

Modelos de Inteligencia Artificial

Sistemas que razonan con reglas y conocimiento, lenguaje natural, robótica y marco ético y legal.

Sesión 1 · 4 horas · Módulo 5071

DEBATE DE APERTURA

¿Qué es la inteligencia?

Sin respuesta correcta. Escribe tu definición en una sola frase.

VOCES DE LA IA · 1 DE 4

«Propongo considerar la pregunta: ¿pueden pensar las máquinas?»

Alan Turing, 1950

Computing Machinery and Intelligence

VOCES DE LA IA · 2 DE 4

«La inteligencia es la parte computacional de la capacidad de lograr objetivos en el mundo.»

John McCarthy, Stanford

What Is Artificial Intelligence?

VOCES DE LA IA · 3 DE 4

«La pregunta de si un ordenador puede pensar no es más interesante que la de si un submarino puede nadar.»

Edsger Dijkstra (atribuida)

Fuente primaria por verificar antes de citarla como literal

VOCES DE LA IA · 4 DE 4

«Igual que la electricidad transformó casi todo hace 100 años, hoy me cuesta pensar en una industria que no vaya a transformar la IA en los próximos años.»

Andrew Ng, 2017

Stanford Graduate School of Business

SESIÓN 1 · HISTORIA DE LA IA

De los orígenes a los sistemas expertos

1950

Turing

Plantea si una máquina puede pensar y propone su famoso test.

1956

Dartmouth

Nace el término «inteligencia artificial» como campo de estudio.

1966

ELIZA

Un chatbot de reglas simula una conversación sin entender nada.

70-80

Sistemas expertos

Reglas de especialistas, como MYCIN en medicina.

Inviernos de la IA: épocas en las que las promesas no se cumplieron y cayeron la financiación y el interés.

SESIÓN 1 · HISTORIA DE LA IA

Del aprendizaje profundo a hoy

1997

Deep Blue

Vence a Kasparov al ajedrez con búsqueda y reglas.

2012

AlexNet

Una red convolucional gana ImageNet y arranca el aprendizaje profundo.

2016

AlphaGo

Gana a Lee Sedol al go, aprendiendo de partidas.

2017

Transformer

La arquitectura en la que se basan los modelos de lenguaje actuales.

2022

ChatGPT

La IA generativa llega al público general.

Qué cambió: más datos, más potencia de cálculo y mejores algoritmos. No una nueva forma de pensar.

SESIÓN 1 · HISTORIA DE LA IA

ChatGPT, Gemini o Claude: ideas de hace décadas

Hoy	Antecedente	Qué ha cambiado
Chat conversacional	ELIZA, 1966, con reglas	Ahora aprende de datos, no de reglas escritas a mano
Predecir la siguiente palabra	Modelos estadísticos del lenguaje, Shannon, 1948	Escala enorme y mucho más contexto
Redes neuronales profundas	Perceptrón, 1958, y retropropagación, años 80	Más datos y procesadores gráficos
Mejorar con valoraciones	Programa de damas de Samuel, 1959, que aprende jugando	Hoy se ajusta con valoraciones de personas
Buscar y luego responder	Recuperación de información y sistemas expertos	Se combina búsqueda con generación de texto

Idea clave: no son ideas nuevas, sino ideas antiguas con muchos más datos y cálculo.

SESIÓN 1 · ENLACE CON BIG DATA

Del Big Data a la IA: un mismo hilo

1 · Datos masivos

Volumen, variedad y velocidad. Por ejemplo, eventos como los de Wikimedia.

→

2 · Procesar

MapReduce, Spark y Databricks limpian y unen datos repartidos.

→

3 · Almacenar

Parquet y Delta guardan en columnas y con versiones.

→

4 · Aprender

Los modelos se entrenan con esos datos y predicen sobre otros nuevos.

Ventanas y medias móviles (Spark, Flink): crean variables a partir de series temporales.

Streaming: permite detectar anomalías casi en tiempo real.

AVISO

La IA no es Skynet

A día de hoy, la IA son sistemas que aprenden patrones de los datos para predecir. Son muy útiles, pero especializados y sin objetivos propios. Lo que da problemas son sus errores y su mal uso, no una rebelión.

CASO JUEGOS · 1 DE 7

Los juegos, laboratorio de la IA

Durante décadas, la forma más visible de medir la IA ha sido ponerla a jugar contra los mejores humanos.

1997

Ajedrez

Deep Blue, de IBM, gana a Kasparov, campeón del mundo.

2016

Go

AlphaGo, de DeepMind, gana 4–1 a Lee Sedol.

2019

StarCraft II

AlphaStar, de DeepMind, llega a Gran Maestro.

¿Por qué juegos? Reglas claras, un resultado que se mide sin discusión y un rival humano con el que comparar.

CASO JUEGOS · 2 DE 7

Deep Blue contra Kasparov

4–2

1996: gana Kasparov

3½–2½

Mayo de 1997, Nueva York: gana Deep Blue

200 M

jugadas analizadas por segundo

Cómo funcionaba

Hardware diseñado para el ajedrez que exploraba muchísimas jugadas posibles y las puntuaba con criterios escritos por expertos. No aprendía jugando.

Era imbatible en una sola cosa: no sabía hacer nada más que jugar al ajedrez.

CASO JUEGOS · 3 DE 7

AlphaStar: aprender jugando a StarCraft II

Estrategia en tiempo real: parte del mapa está oculta, todo pasa a la vez y hay que decidir cientos de acciones por minuto.

10–0
contra los profesionales TLO y MaNa (dic. 2018)

200 años
de partidas jugadas por cada agente

14 días
de entrenamiento en una liga entre agentes

La diferencia con Deep Blue: nadie le escribió cómo jugar. Primero imitó partidas humanas y después mejoró jugando contra versiones de sí mismo: **aprendizaje por refuerzo**, que veremos en la sesión 2.

CASO JUEGOS · 4 DE 7

¿Jugaba en igualdad de condiciones?

Enero de 2019

Veía todo lo visible del mapa sin mover la cámara, algo que un humano no puede hacer.

En ráfagas cortas hacía más acciones por minuto que los profesionales.

En directo, MaNa ganó a una versión que sí tenía que mover la cámara.

Octubre de 2019

Vista de cámara como un humano y un máximo de 22 acciones cada 5 segundos.

Partidas anónimas en el servidor oficial, contra jugadores reales.

Gran Maestro con las tres razas: por encima del 99,8 % de los jugadores. Publicado en Nature.

Una comparación con humanos solo vale si las condiciones son las mismas.

CASO JUEGOS · 5 DE 7

Faker: lo que AlphaStar tenía gratis

El mejor jugador de la historia de League of Legends, otro juego de estrategia en tiempo real.

30 años
y sigue en la élite con T1

6
campeonatos del mundo, de 2013 a 2025

3–2
a KT Rolster en la final de 2025

Su pantalla parpadea sin parar: salta con la cámara a la zona de un compañero para ver qué pasa y vuelve a la suya en una fracción de segundo.

▶ Ver el vídeo en X

Idea clave: esa visión global, que a él le cuesta años de práctica, es la que AlphaStar tenía sin esfuerzo y DeepMind tuvo que quitarle.

CASO JUEGOS · 6 DE 7

Trackmania: aprender sin datos humanos

Juego de carreras. El bot empieza conduciendo al azar y solo recibe una recompensa cuando va más rápido: nadie le enseña partidas de personas.

Yosh · circuito A01

23,75 s

frente a 23,79 s, el récord humano que llevaba dos años sin batirse. Más de 3.000 intentos y unas 400 horas.

Linesight · proyecto abierto

10 de 12

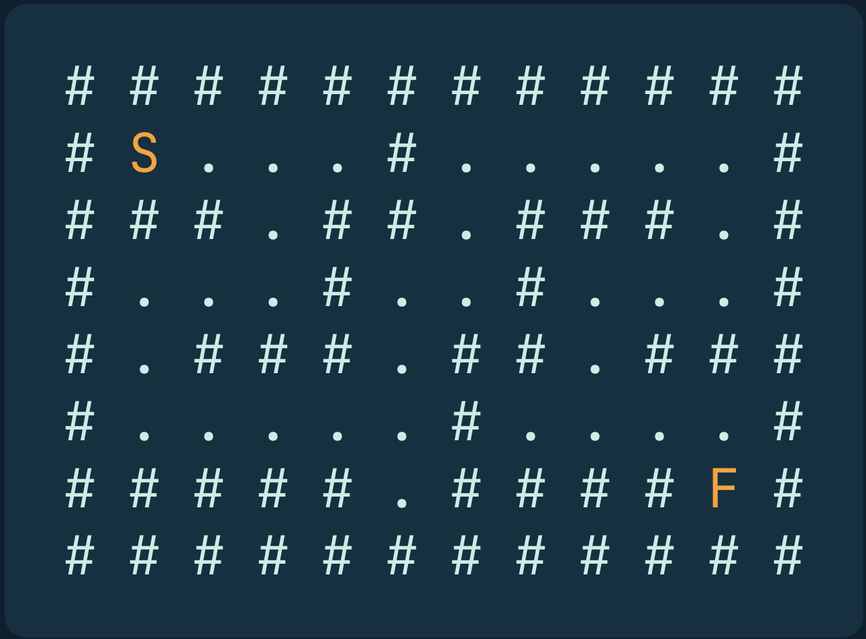
récords mundiales de la campaña oficial de Trackmania Nations Forever, batidos en 2024.

No siempre necesitan datos humanos, pero sí muchísima experiencia. Si no hay un dataset, la generan ellos mismos jugando una y otra vez.

AlphaZero (2018) hizo lo mismo en ajedrez: solo las reglas y partidas contra sí mismo.

Fuentes: Dexerto · Linesight · AlphaZero en Science

Un coche que aprende solo el circuito



El reto
Ir de la salida **S** a la meta **F**. Hay muros # y callejones sin salida.

Sin dataset
Nadie le enseña el camino. Empieza al azar y solo recibe premios y castigos.

El material
Notebook **06_IA_Aprendizaje_Refuerzo** del repositorio, en Databricks o en local.

Las cinco piezas del aprendizaje por refuerzo

Pieza	Qué es	En el notebook
Agente	Quien decide	El coche · elegir_accion()
Entorno	El mundo que responde	El circuito · mover()
Estado	Dónde está ahora	La casilla: (fila, columna)
Acción	Lo que puede hacer	↑ ↓ ← →
Recompensa	La nota de cada acción	−1 por paso · −5 por choque · +100 en la meta

El ciclo: el agente elige una acción → el entorno le devuelve la nueva casilla y una recompensa → aprende de ello → vuelve a empezar.

El modelo: qué es Q-learning

Un método de aprendizaje por refuerzo que aprende, para cada situación, cuánto vale cada acción posible.

`Q(casilla, movimiento)`

= los puntos que espera ganar si hace ese movimiento ahora y después juega lo mejor posible.

La Q viene de *quality*: la calidad de una acción en una situación.

No necesita el mapa

No sabe dónde están los muros ni la meta: lo descubre probando.

Aprende en cada paso

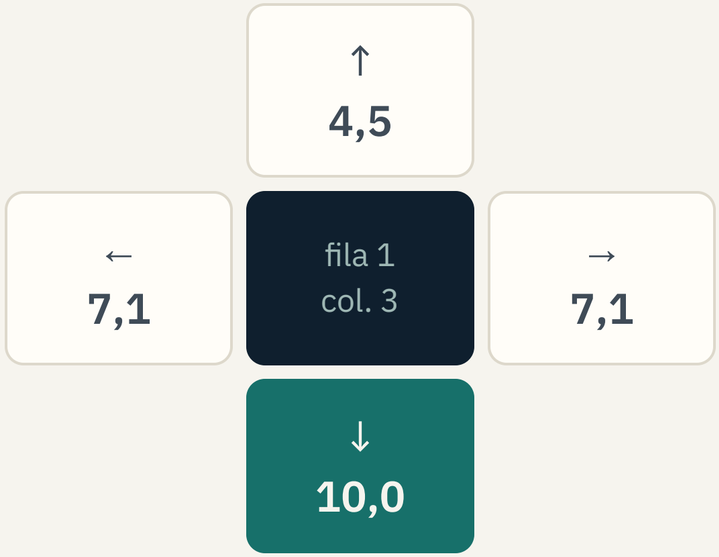
No espera a terminar el intento: corrige tras cada movimiento.

Aprende lo mejor

Aunque a veces se mueva al azar, la tabla apunta a la mejor ruta.

La tabla Q por dentro

Una casilla: el cruce junto a la salida

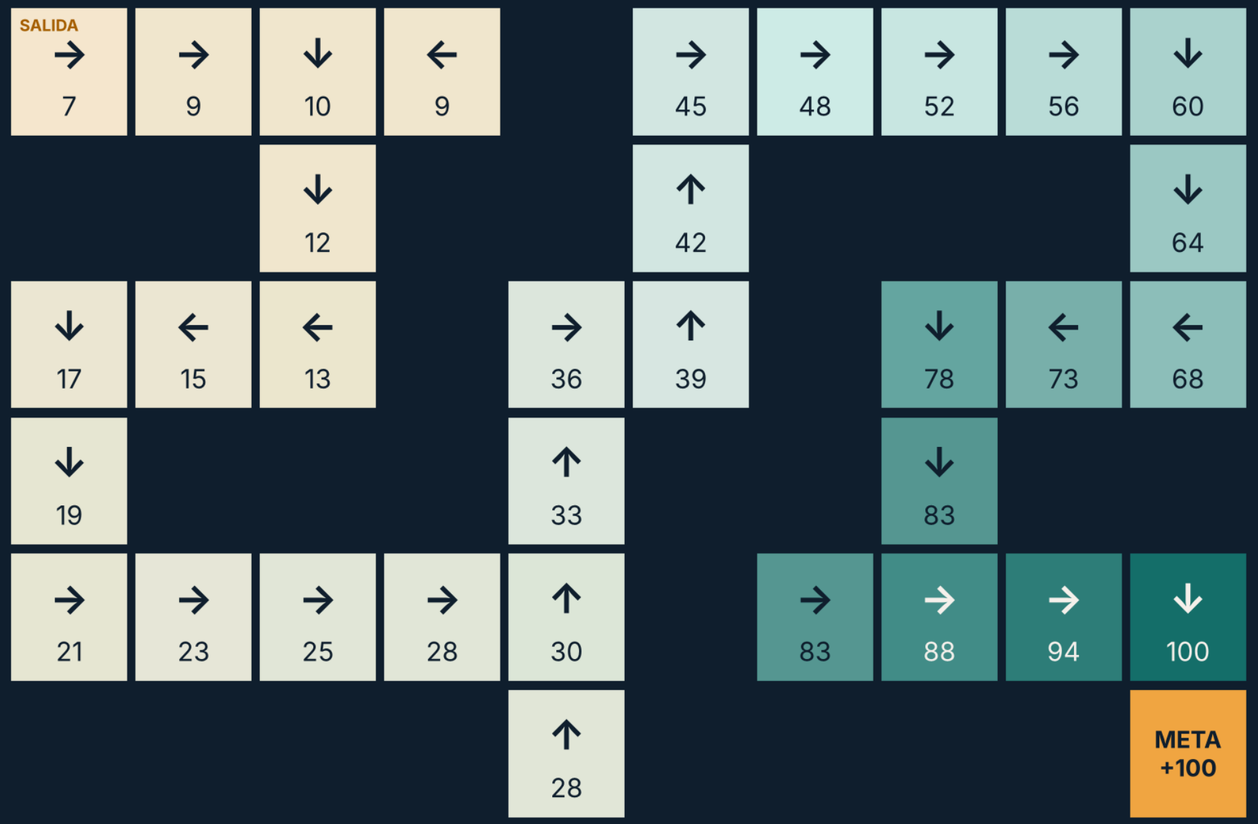



```
q = np.zeros((8, 12, 4))
# 8 filas × 12 columnas × 4 movimientos
q[1, 3] → [4.5, 10.0, 7.1, 7.1]
q[1, 3].argmax() → 1 (↓, el mejor)
q[1, 3].max() → 10.0 (lo que vale estar aquí)
```

384 números, aunque solo 33 casillas son pista: los huecos de los muros nunca se usan.

Antes de entrenar todo vale 0: hay empate en todo y elige al azar.

La tabla entera: el valor viaja desde la meta



El número
Lo que vale estar en esa casilla: 100 junto a la meta, 7 en la salida.

La flecha
Su mejor movimiento. Siguiendo las flechas se obtiene la ruta óptima.

Cada paso atrás vale **−1 + 0,95 × el siguiente**: 100 → 94 → 88 → 83…

Cuando la tabla no cabe: redes neuronales

Nuestro ejemplo

Tabla Q

El estado es una casilla: 33 posibles.

Una fila por casilla, 4 valores en cada una.

Solo sabe de casillas que ha visitado.

Atari, Trackmania

Red neuronal

El estado es la imagen de la pantalla: combinaciones casi infinitas.

La red recibe la imagen y calcula los valores Q de cada acción.

Generaliza a situaciones que nunca ha visto.

DQN (DeepMind, 2015): Q-learning con una red neuronal. Jugó a 49 juegos de Atari viendo solo los píxeles y la puntuación, y superó a los métodos anteriores en 43.

La regla que aprende, en una línea

```
q[pos][accion] += ALFA * (recompensa + GAMMA * futuro - q[pos][accion])
```

1 · Lo que esperaba

El valor actual de la tabla para esa casilla y ese movimiento.

2 · Lo que ha pasado

La recompensa recibida más lo mejor que espera desde la casilla nueva.

3 · Corrige

Mueve el valor hacia lo que ha pasado, solo una parte: ALFA.

Ejemplo: primer choque contra un muro. Esperaba 0, recibe −5 y el futuro vale 0 → nuevo valor = 0 + 0,5 × (−5 − 0) = **−2,5**. La próxima vez evitará ese movimiento.

Explorar o aprovechar lo aprendido

Explorar

Probar un movimiento al azar. Sirve para descubrir caminos que todavía no conoce.

Aprovechar

Elegir el movimiento con más valor en la tabla. Sirve para usar lo que ya sabe.

Probabilidad de moverse al azar a lo largo del entrenamiento:

≈100 %	50 %	5 %
intento 1: todo al azar	intento 150	del intento 300 al final

Celda 1: los parámetros

Parámetro	Valor	Qué controla
EPISODIOS	500	Cuántos intentos de entrenamiento
MAX_PASOS	300	Pasos máximos por intento antes de cortarlo
ALFA	0,5	Cuánto corrige con cada experiencia nueva
GAMMA	0,95	Cuánto valora las recompensas futuras
EXPLORACION_MINIMA	0,05	Azar que nunca desaparece
RECOMPENSA_PASO	−1	Cada paso cuesta: premia ir rápido
RECOMPENSA_CHOQUE	−5	Castigo por chocar contra un muro
RECOMPENSA_META	+100	Premio por llegar a la meta
SEMILLA	42	Repite el mismo azar en cada ejecución

Celda 2: las cuatro funciones

`mover(pos, accion)`

El entorno. Aplica el movimiento y devuelve la casilla nueva, la recompensa y si ha terminado.

`elegir_accion(q, pos, exploracion)`

El agente. Al azar con probabilidad «exploracion»; si no, el movimiento con más valor.

`jugar_episodio(q, exploracion, aprender)`

Un intento completo. Repite elegir y mover hasta la meta o MAX_PASOS; si aprender=True, actualiza la tabla.

`dibujar(camino, titulo)`

Para verlo. Pinta el circuito y marca con * las casillas por las que ha pasado.

jugar_episodio() devuelve el camino, la recompensa total, si llegó y cuántas veces chocó.

El notebook, celda a celda

#	Celda	Qué hace
1	**Parámetros**	El circuito y los ajustes del entrenamiento
2	**Entorno y agente**	Las cuatro funciones; aún no ejecuta nada
3	**Entrenamiento**	500 intentos; guarda pasos, choques y recompensa de cada uno
4	**Cómo conducía**	El circuito recorrido en los intentos 1, 25 y 100
5	**Resultado final**	Conduce sin azar y muestra el mapa de flechas aprendido
6	**Curva de aprendizaje**	Gráfica de pasos por intento, con la media de 20
7	**Historial en Spark**	Los 500 intentos como DataFrame, con display()

Para experimentar: cambia la celda 1 y vuelve a ejecutar desde ella (Run all below).

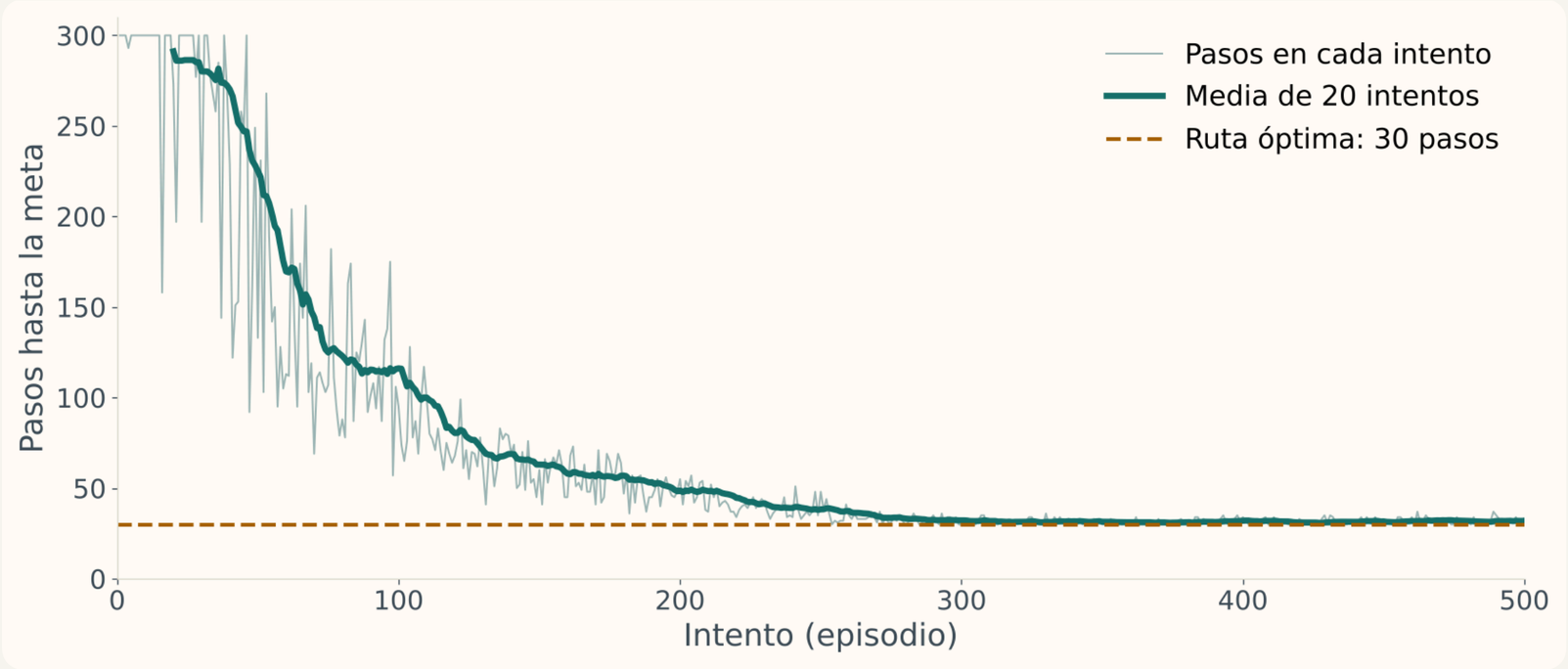
Resultados: de perderse a la ruta óptima

Intento 1
300 pasos, 162 choques, no llega

Intento 100
95 pasos, 29 choques

Intento 300 en adelante
≈30 pasos, casi sin choques

Prueba final, sin azar
30 pasos, 0 choques



Experimentos para la clase

`EPISODIOS = 20`

No llega a la meta: da vueltas hasta agotar los 300 pasos. Le falta experiencia.

`RECOMPENSA_CHOQUE = 0`

Aprende a chocar sin parar: 300 choques. Chocar sale gratis y moverse cuesta −1.

`SEMILLA = 1, 7, 123`

Otro azar, mismo destino: siempre acaba en la ruta de 30 pasos.

La lección de Klarna, otra vez: el agente optimiza exactamente lo que medimos, no lo que queríamos.

Del juguete a Trackmania: aquí basta una tabla de 384 números. Allí el estado es casi infinito y la tabla la sustituye una red neuronal.

CASO JUEGOS · 7 DE 7

Lo que enseñan los juegos

1

Necesitan muchísima experiencia: partidas humanas o millones de intentos propios.

2

Son especialistas: el campeón de un juego no sabe jugar a otro.

3

Ver más o pulsar más rápido es ventaja, no inteligencia.

4

Superan a humanos en tareas acotadas y medibles. Ni Skynet ni magia.

Pregunta para el grupo: ¿qué tarea de vuestro trabajo se parece a un juego, con reglas claras y un resultado que se pueda medir?

SESIÓN 1 · HISTORIA Y FUNDAMENTOS

De ganar partidas a predecir palabras

Todos estos sistemas hacen lo mismo en el fondo: mirar la situación y estimar qué es lo mejor o lo más probable a continuación.

1997 · Deep Blue

¿Qué jugada es mejor?

Puntúa posiciones con reglas escritas por expertos.

→

2019 · AlphaStar

¿Qué acción me hará ganar?

Aprende de partidas a dar probabilidades a cada acción.

→

Hoy · Modelos de lenguaje

¿Qué palabra viene después?

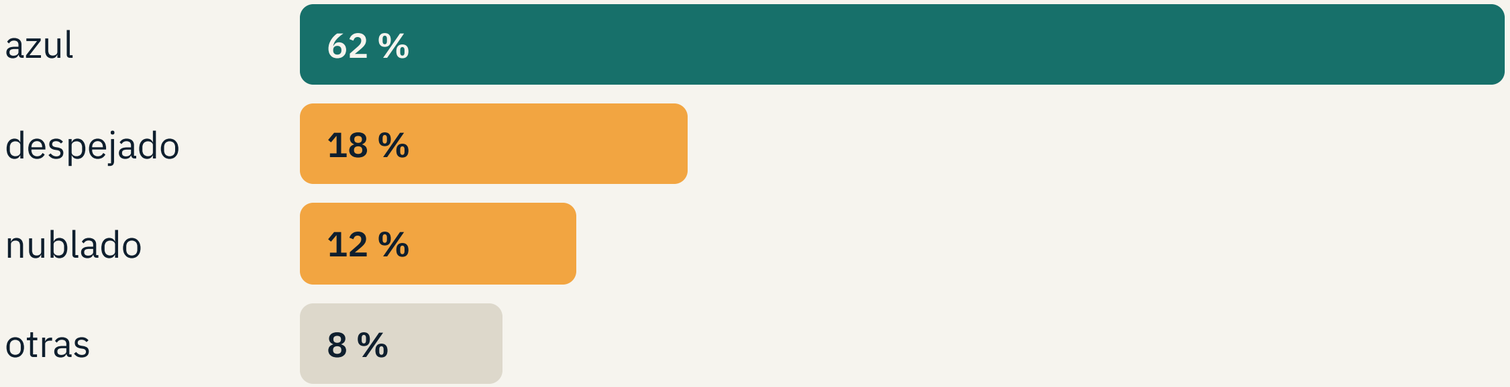
Aprenden de enormes cantidades de texto escrito por personas.

El salto: el lenguaje no tiene marcador de victoria, pero sí muchísimos ejemplos. Predecir bien la siguiente palabra resultó ser un objetivo suficiente.

SESIÓN 1 · HISTORIA Y FUNDAMENTOS

Predicción estadística, no comprensión

Un modelo de lenguaje calcula qué palabra es más probable a continuación: «El cielo está…».



Idea clave: responde con lo más probable según los patrones de sus datos de entrenamiento. Por eso puede sonar seguro y equivocarse.

SESIÓN 1 · CÓMO VE EL TEXTO UN MODELO

De «El cielo está…» a números

1 · Texto

«El cielo está»

2 · Tokens

`El | cielo | está`

`412 · 8830 · 97`

3 · Vectores

`cielo`

`[0,2 -0,7 0,4 …]`

4 · Modelo

Calcula la probabilidad de cada token siguiente.

5 · Salida

azul 62 %
despejado 18 %
nublado 12 %

Token: una palabra, un trozo de palabra o un signo. No siempre coincide con una palabra.

Vector: lista de números que sitúa el significado; palabras parecidas quedan cerca.

SESIÓN 1 · CÓMO VE EL TEXTO UN MODELO

Los tokens: el texto troceado

Frase: «El cielo está despejado hoy»

El	cielo	está	desp	ejado	hoy
412	8830	97	15204	3381	2210

Token no es palabra

«despejado» se parte en dos piezas.

Vocabulario fijo

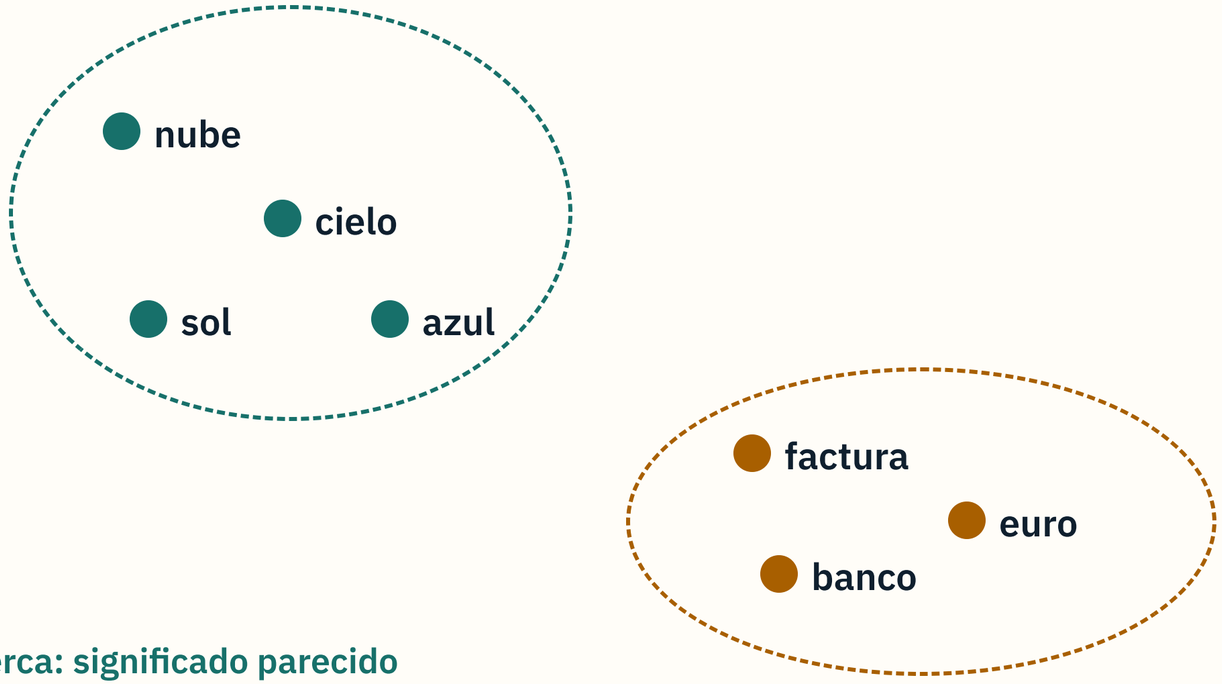
Cada pieza tiene un número en una tabla del modelo.

Se cuenta en tokens

Longitud máxima y coste suelen medirse así.

Los vectores: el significado como mapa

Mapa de significados en 2 dimensiones (ejemplo)



Cerca: significado parecido

Cada palabra es un punto. Su posición son sus números.

Cerca, parecido. «cielo» queda junto a «nube» y lejos de «factura».

```
cielo → [0,2 -0,7 0,4 …]
```

Aquí 2 números; en un modelo real, cientos o miles.

SESIÓN 1 · CÓMO VE EL TEXTO UN MODELO

Bases de datos vectoriales

1 · Documentos
Textos troceados en fragmentos.

→

2 · Vectores
Un modelo de incrustaciones convierte cada fragmento en un vector.

→

3 · Base vectorial
Guarda los vectores y busca los más cercanos a una consulta.

→

4 · Modelo
Recibe los fragmentos hallados como contexto y responde.

Consulta: «cielo» frente a…	Similitud (ejemplo inventado)
nube	0,86
azul	0,81
factura	0,05

Ojo: la base vectorial no genera texto, solo encuentra lo parecido. Esta técnica se llama RAG: buscar y luego generar.

Un ETL de siempre, con dos pasos nuevos

ETL clásico: datos para informes



Pipeline para IA: documentos para un asistente



Lo nuevo son dos pasos (en naranja). El resto es Big Data de siempre: Spark, tablas Delta, cargas programadas y control de calidad.

Trocear el texto con LangChain

```
from langchain_text_splitters import (
RecursiveCharacterTextSplitter)

splitter = RecursiveCharacterTextSplitter(
chunk_size=1000,
chunk_overlap=200,
)
trozos = splitter.split_text(texto)

# Medir en tokens en vez de caracteres:
RecursiveCharacterTextSplitter.from_tiktoken_encoder(
model_name="gpt-4", chunk_size=300)
```

`chunk_size`
Tamaño máximo de cada trozo; por defecto, en caracteres.

`chunk_overlap`
Solape entre trozos seguidos, para no partir una idea por la mitad.

«Recursive»
Corta primero por párrafos, luego por líneas y luego por espacios.

Trocear no es tokenizar: LangChain corta el documento en fragmentos; los tokens los crea después, por dentro, el modelo de embeddings.

Después de trocear: tokenizar y vectorizar

1 · Un trozo de texto

«El Sporting ganó al Valencia en El Molinón.»

2 · El tokenizador del modelo lo parte en 14 tokens

`[CLS]` `el` `sporting` `gan` `##o` `al` `valencia` `en` `el` `mo` `##lino` `##n` `.` `[SEP]`

→ 101 3449 7419 25957 2080 2632 13083 …

3 · El modelo devuelve un vector de 384 números

`[-0,030 0,085 -0,012 0,031 -0,096 …]`

```
from langchain_huggingface import (
HuggingFaceEmbeddings)
emb = HuggingFaceEmbeddings(model_name=
"sentence-transformers/all-MiniLM-L6-v2")
vector = emb.embed_query(trozo)
# los pasos 2 y 3 ocurren aquí dentro
```

Cada modelo, su tokenizador

Este, entrenado en inglés, quita tildes y mayúsculas. El de GPT-4 lo parte en 12 tokens distintos.

Máximo 256 tokens

Lo que sobra se corta sin avisar: por eso troceamos antes.

Los documentos en la arquitectura medallion

Bronce

Los documentos tal cual llegan: PDF, HTML, correos. Se guardan en un volumen, con su origen y fecha.

`doc_id · ruta`
`origen · fecha_carga`

→

Plata

Texto extraído, limpio y troceado con LangChain: una fila por fragmento.

`chunk_id · doc_id`
`texto · posicion`

→

Oro

Fragmentos listos para buscar: con su vector y los metadatos para filtrar.

`chunk_id · texto`
`embedding · metadatos`

→

Índice vectorial

Sincronizado con la tabla oro. Lo consulta el asistente RAG.

Igual que con tablas: si cambia el troceado o el modelo, se rehace plata u oro desde bronce sin volver a descargar nada.

Trazabilidad: cada respuesta del asistente se puede seguir hasta el documento original por su doc_id.

SESIÓN 1 · VECTORES Y BIG DATA · 5 DE 6

RAG: buscar antes de responder

Retrieval-Augmented Generation: el modelo responde con la información que encontramos en nuestros documentos, no solo con lo que aprendió.

1 Llega una pregunta → **2** Se vectoriza con el mismo modelo → **3** Se buscan los trozos más parecidos → **4** Se montan en el prompt → **5** El modelo responde citando

```
Instrucción: responde solo con el contexto; si no está, di que no lo sabes.
Contexto: [doc 12] «Las devoluciones se aceptan durante 30 días…» [doc 40] …
Pregunta: ¿Cuántos días tengo para devolver un pedido?
```

Datos propios
Sin reentrenar el modelo.

Al día
Basta con actualizar el índice.

Comprobable
Cada respuesta cita sus fuentes.

Pero…
Si la búsqueda falla, la respuesta también.

SESIÓN 1 · VECTORES Y BIG DATA · 6 DE 6

El bucle RAG dentro de la arquitectura de datos



El índice une los dos mundos: la carga lo llena con el lakehouse de siempre; la consulta lo lee en milisegundos.

Cierra el bucle: preguntas, respuestas y valoraciones son datos nuevos que vuelven a bronce.

SESIÓN 1 · FUNDAMENTOS

¿Qué es un sistema inteligente?

Un sistema que percibe su entorno, decide qué hacer para lograr un objetivo y actúa en consecuencia.

1 · Percibe

Recibe datos: sensores, texto, imágenes, registros de una base de datos.

2 · Decide

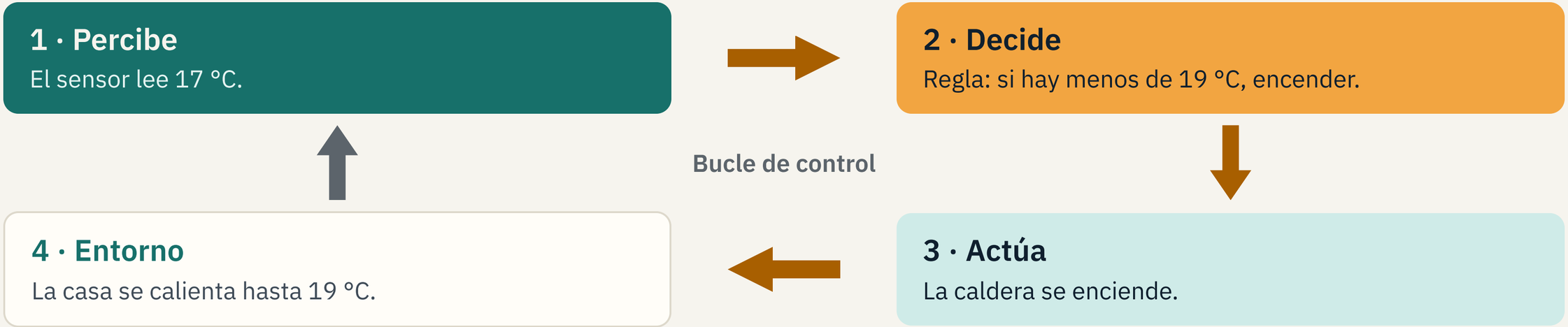
Razona con reglas, con probabilidades o con lo que ha aprendido de ejemplos.

3 · Actúa

Devuelve una respuesta, una alerta o una acción sobre el entorno.

SESIÓN 1 · FUNDAMENTOS

Ejemplo: el termostato



Regla fija, sin aprender: el ciclo cumple percibir-decidir-actuar, pero nunca cambia su regla.

SESIÓN 1 · FUNDAMENTOS

IA débil e IA fuerte

Aspecto	IA débil	IA fuerte
Alcance	Una tarea concreta	Inteligencia general, como la humana
Situación	Es la que se usa hoy	Hipotética: aún no existe
Ejemplos	Filtro de spam, traductor, recomendador	Solo en ciencia ficción
Ventaja	Precisa y rentable en su tarea	Resolvería cualquier problema
Límite	No generaliza fuera de su tarea	Riesgos y control sin resolver

SESIÓN 1 · HACIA DÓNDE VA

IA autónoma: de asistente a agente

Asistente

Responde a lo que se le pide. La persona decide y actúa.

Agente

Planifica pasos y usa herramientas. La persona supervisa.

Autónoma

Actúa sola durante largos periodos. Hoy, solo en entornos acotados.

Riesgos: errores en cadena, permisos excesivos y falta de responsable claro.

Medida: límites de permisos, registro de acciones y supervisión humana.

Asistente: responde, la persona decide

Persona
Hace una pregunta o pide un borrador.

→

IA
Contesta o propone.

→

Persona
Revisa, decide y actúa.

Cuándo se usa

Atención al cliente
Responde preguntas frecuentes sobre pedidos y devoluciones.

Redacción
Propone un borrador de correo que la persona revisa y envía.

Programación
Sugiere código que el desarrollador acepta o rechaza.

Control humano: alto, en cada respuesta.

Riesgo principal: una respuesta errónea dicha con seguridad.

CASO KLARNA · 1 DE 3

Klarna: un asistente que atiende a los clientes

Febrero de 2024: la empresa de pagos presentó los resultados de su asistente de IA en atención al cliente.

2,3 M
conversaciones en el primer mes

⅔
de los chats atendidos por la IA

< 2 min
de resolución media

700
personas de trabajo equivalente, según Klarna

Encaje en la sesión: es un **asistente** (responde a peticiones), no un equipo de agentes que planifica.

Matiz: todas las cifras las dio la propia empresa; la prensa no las verificó.

Lo que se midió: volumen, rapidez y coste. ¿Y la calidad?

CASO KLARNA · 2 DE 3

Mayo de 2025: la IA se queda, pero vuelven las personas

El coste fue un factor «demasiado predominante» y eso llevó a «menor calidad».

Sebastian Siemiatkowski, consejero delegado de Klarna, a Bloomberg, según Customer Experience Dive · traducción propia

Qué anunció
Invertir en la calidad del soporte humano, y que siempre habrá una persona si el cliente la quiere.

Qué hizo
Empezó a contratar personal de atención al cliente en remoto.

Qué mantiene
La IA sigue atendiendo dos tercios de las consultas; modelo mixto de IA y personas.

Según la empresa: tiempos de respuesta un 82 % mejores desde el lanzamiento y un 25 % menos de consultas repetidas.

CASO KLARNA · 3 DE 3

Lo que enseña el caso

1

Lo que se mide es lo que se optimiza: coste y rapidez no son calidad.

2

La IA sirve para lo repetitivo; los casos delicados piden personas.

3

Hay que poder llegar a una persona cuando se quiere.

4

Las cifras de éxito de una empresa sobre su propio sistema hay que contrastarlas.

Mismo patrón que Tesla y los agentes: más automatización exige mejor supervisión y mejores métricas.

Pregunta para el grupo: si fuerais Klarna, ¿qué medirías además del coste y del tiempo de respuesta?

Agente: planifica y usa herramientas

Persona
Fija el objetivo.

→

IA
Planifica los pasos.

→

Herramientas
Buscar, consultar, escribir.

→

Persona
Aprueba lo importante.

Cuándo se usa

Soporte técnico

Lee una incidencia, consulta el inventario y abre un ticket; un técnico lo aprueba.

Investigación

Busca fuentes, las compara y entrega un informe con enlaces para revisar.

Administración

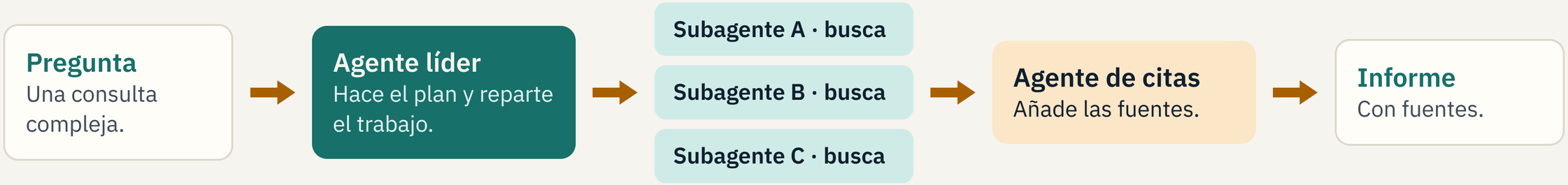
Reúne datos de varias hojas y prepara una conciliación para validar.

Control humano: medio, en los pasos críticos.

Riesgo principal: errores en cadena y permisos excesivos.

CASO DE AGENTES · 1 DE 15

Un equipo de agentes que investiga en paralelo



Quién
El sistema «Research» de Anthropic, descrito en su blog de ingeniería.

Patrón
Orquestador y trabajadores: un líder coordina y varios subagentes ejecutan a la vez.

Flujo de datos
Resultados de búsquedas web, leyendo y resumiendo; es texto, no Big Data tipo Spark.

CASO DE AGENTES · 2 DE 15

Con qué trabajan: herramientas y memoria

Búsqueda web
La fuente principal; se usa varias veces, afinando la consulta.

Integraciones
Herramientas externas, por ejemplo Google Workspace y servidores MCP.

Memoria
El líder guarda su plan para no perderlo cuando el contexto se trunca.

Pensamiento extendido
Un borrador para planificar y juzgar la calidad de lo encontrado.

Archivos externos
Los subagentes guardan resultados y devuelven solo una referencia.

Agente probador
Reescribe descripciones de herramientas defectuosas: –40 % de tiempo.

Idea clave: un agente es un modelo más herramientas y memoria; sin ellas, solo conversa.

AGENTES · MCP · 1 DE 4

MCP: el «USB-C» de las aplicaciones de IA

Model Context Protocol: un estándar abierto para conectar aplicaciones de IA con herramientas y datos.

Sin un estándar

3 × 3 = 9

Tres aplicaciones y tres herramientas: nueve integraciones a medida.

Con MCP

3 + 3 = 6

Cada herramienta se publica una vez y la usan todas las aplicaciones.

Host
La aplicación: Claude, VS Code, un agente propio.

→

Cliente MCP
Dentro del host: una conexión por servidor.

→

Servidor MCP
Expone herramientas y datos: GitHub, una base de datos…

AGENTES · MCP · 2 DE 4

Qué ofrece un servidor MCP y cómo se usa

Herramientas
Acciones: consultar, crear, calcular.
Las elige el modelo.

Recursos
Datos para leer: un fichero, una tabla.
Los elige la aplicación.

Prompts
Plantillas de petición ya preparadas.
Los elige el usuario.

Una llamada, paso a paso

1 · Descubrir ¿qué herramientas hay? → **2 · Elegir** el modelo decide cuál → **3 · Llamar** con sus argumentos → **4 · Ejecutar** el servidor responde → **5 · Usar** el modelo responde

Local (stdio): el host arranca el servidor en el mismo ordenador.

Remoto (HTTP): el servidor vive en la nube, como un servicio web.

AGENTES · MCP · 3 DE 4

Los riesgos de conectar herramientas

Servidores de terceros

Un servidor local es código que corre con tus permisos.

→ Solo de fuentes fiables, y aislado.

Instrucciones escondidas

En la descripción de una herramienta o en los datos que devuelve.

→ Revisar servidores y filtrar con guardrails.

Permisos de más

Un token con acceso a todo: si se filtra, se pierde todo.

→ Mínimo privilegio, permisos por operación.

Tokens prestados

Un servidor que reenvía credenciales emitidas para otro servicio.

→ La especificación lo prohíbe expresamente.

Un punto de control en medio: AgentCore Gateway y Policy en AWS, o el AI Broker de Zscaler.

AGENTES · MCP · 4 DE 4 · PRÁCTICA

Un servidor MCP con los datos del Sporting

```python
from mcp.server import MCPServer
mcp = MCPServer("sporting-2015-16")

@mcp.tool()
def partido(jornada: int) → dict:
"""Detalle de un partido por jornada."""
...

@mcp.resource("sporting://partidos")
def todos_los_partidos() → str: ...

@mcp.prompt()
def cronica_partido(jornada: int) → str: ...

mcp.run() # stdio
```

1 · Instalar

```
pip install "mcp[cli]"
```

2 · Probar sin IA

```
python 07_IA_Servidor_MCP_Sporting.py --probar
```

Resultado real

`resumen_temporada()` → 38 partidos:
10 V · 9 E · 19 D · 39 puntos

3 · Conectarlo a un asistente

Inspector (`mcp dev`) o Claude Desktop

El docstring es lo que lee el modelo para decidir cuándo usar la herramienta: escribirlo bien también es afinar.

CASO DE AGENTES · 3 DE 15

Cómo reparte el trabajo el líder

Cada subagente recibe cuatro cosas

1 · Objetivo

2 · Formato de salida

3 · Qué herramientas usar

4 · Límites de la tarea

El esfuerzo se ajusta a la dificultad

Consulta	Agentes	Llamadas a herramientas
Buscar un dato	1	3 a 10
Comparar opciones	2 a 4 subagentes	10 a 15 cada uno
Investigación compleja	Más de 10 subagentes	Repartidas por tema

Estrategia: empezar con consultas amplias y cortas, y estrechar poco a poco.

CASO DE AGENTES · 4 DE 15

Cómo se comprueba que funciona

~20

Consultas reales

Para empezar bastó una muestra pequeña sacada del uso real.

Un modelo hace de juez

Puntúa de 0,0 a 1,0 cinco criterios:

- precisión de los datos
- precisión de las citas
- completitud
- calidad de las fuentes
- eficiencia con las herramientas

Personas

Detectaron lo que la automatización no veía: preferían webs hechas para buscadores.

Conexión con la sesión 2: es la idea de evaluar con datos de prueba, aunque aquí el juez es otro modelo.

Ojo: se mide si el resultado es correcto, no el camino que siguió cada agente.

CASO DE AGENTES · 5 DE 15

Qué se gana y cuánto cuesta

+90,2 %

frente a un solo agente, en su evaluación interna de investigación

hasta −90 %

de tiempo en consultas complejas, por trabajar en paralelo

El precio: tokens consumidos



Solo compensa en tareas de mucho valor.

Matiz: son cifras de la propia empresa y de una evaluación interna; no es una auditoría independiente.

CASO DE AGENTES · 6 DE 15

Llevarlo a producción es otro problema

Estado y errores
Un agente recuerda lo que hace, y los fallos se acumulan. Se retoma desde puntos de control, sin empezar de cero.

Depurar
Cada ejecución es distinta. Se registran las decisiones, sin leer las conversaciones, para proteger la privacidad.

Actualizar
Los agentes trabajan mucho rato, así que las versiones se cambian poco a poco («despliegue arcoíris»).

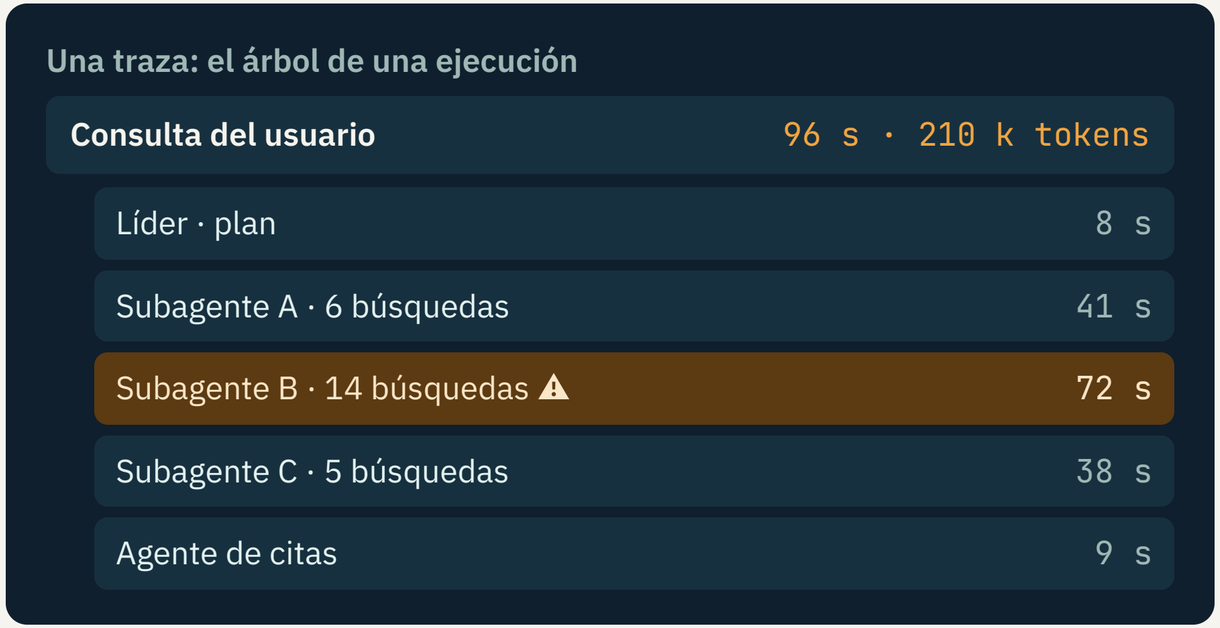
Esperas
El líder espera a que acabe cada tanda de subagentes; trabajar en asíncrono sería más rápido, pero más complejo.

Idea clave: el modelo es solo una parte; el resto es ingeniería de software clásica.

En AWS: AgentCore Runtime ejecuta cada sesión aislada, hasta 8 horas, y AgentCore Memory guarda el estado.

CASO DE AGENTES · 7 DE 15

Monitorizar: ver qué hizo cada agente



Coste y tiempo
Tokens, euros y segundos por ejecución y por agente.

Comportamiento
Cuántos subagentes lanza, cuántas herramientas usa, errores y reintentos.

Calidad
Nota del modelo juez y valoraciones de los usuarios.

En AWS: AgentCore Observability recoge las trazas (OpenTelemetry) en CloudWatch. **En Databricks:** MLflow Tracing las guarda en tablas Delta. Después, cuadros de mando y alertas.

CASO DE AGENTES · 8 DE 15

Configurar el equipo: cuántos, quién y hasta dónde

1 · En el prompt del líder
Reglas de esfuerzo: cuántos subagentes según la dificultad. Así lo hizo Anthropic.

2 · En cada subagente
Cuándo usarlo, sus instrucciones, sus herramientas y su modelo: el líder más potente, los subagentes más baratos.

3 · Límites en el código
Profundidad, subagentes a la vez y presupuesto máximo.

```
agents={
"buscador": AgentDefinition(
description="Busca y resume fuentes
sobre un subtema",
prompt="Eres un investigador...",
tools=["WebSearch", "WebFetch"],
model="sonnet",
maxTurns=15)},
max_budget_usd=5.0,
env={
"CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS": "5",
"CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH": "1"}
```

El prompt orienta; los límites del código obligan. Hacen falta las dos cosas. En AWS, AgentCore Policy añade reglas fijas antes de cada llamada a una herramienta.

CASO DE AGENTES · 9 DE 15

Afinar: un ciclo, no un ajuste único

1 · Medir
~20 consultas, juez y personas

→

2 · Observar
leer las trazas paso a paso

→

3 · Cambiar una cosa
prompt, herramienta o modelo

→

4 · Volver a medir
¿mejora o empeora?

Qué se ajusta	Ejemplo del caso
Cómo delega el líder	Objetivo, formato, herramientas y límites para cada subagente
Reglas de esfuerzo	Dejar de lanzar 50 subagentes para preguntas simples
Descripción de herramientas	Un agente probador las reescribió: –40 % de tiempo
Estrategia de búsqueda	Empezar con consultas amplias y luego estrechar
El propio modelo	Los modelos diagnostican sus fallos y proponen mejoras al prompt

Ojo: afinar un agente casi nunca es reentrenar el modelo. Es ajustar instrucciones, herramientas y límites. **En AWS:** AgentCore Evaluations puntúa las sesiones con jueces automáticos.

El caso montado en AWS

Qué necesita el sistema	Servicio de AWS	Lo vimos en
Los modelos Claude, líder y subagentes	**Amazon Bedrock**	El caso
Ejecutar el agente, cada sesión aislada	**AgentCore Runtime**	Producción
Herramientas: APIs y Lambda como MCP	**AgentCore Gateway**	Herramientas
Guardar el plan y lo aprendido	**AgentCore Memory**	Herramientas
Permisos y reglas fijas	**AgentCore Identity y Policy**	Configurar
Trazas, tiempos y tokens	**AgentCore Observability** + CloudWatch	Monitorizar
El modelo juez	**AgentCore Evaluations**	Evaluación y afinar
Filtrar contenido no deseado	**Bedrock Guardrails**	Guardrails

Las piezas son las mismas en cualquier nube: Azure, Google Cloud o Databricks tienen sus equivalentes. Cambian los nombres, no las ideas.

CASO DE AGENTES · 11 DE 15

Del portátil a AWS: desplegar el agente

```
from bedrock_agentcore.runtime import (
BedrockAgentCoreApp)
from claude_agent_sdk import query

app = BedrockAgentCoreApp()

@app.entrypoint
async def investigar(payload):
async for msg in query(
prompt=payload["prompt"],
options=opciones):
if hasattr(msg, "result"):
return {"informe": msg.result}

app.run()
```

1 · Claude a través de Bedrock
`CLAUDE_CODE_USE_BEDROCK=1`

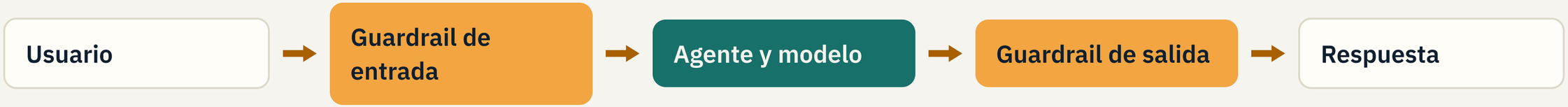
2 · Probar en local
`agentcore dev`

3 · Desplegar en Runtime
`agentcore deploy`

4 · Invocar y mirar las trazas
`agentcore invoke → CloudWatch`

El código del agente no cambia: se envuelve en un punto de entrada y AWS se encarga de servidores, escalado y aislamiento.

Guardrails: filtros a la entrada y a la salida



Qué puede filtrar Amazon Bedrock Guardrails

Contenido dañino
Odio, insultos, violencia… y ataques al prompt.

Temas prohibidos
Los que definamos: asesoría legal, política…

Palabras
Una lista propia: insultos, nombres de la competencia.

Datos personales
Bloquear o enmascarar teléfonos, direcciones…

Anclaje
¿La respuesta se apoya en las fuentes del RAG?

Razonamiento automático
Comprueba la respuesta contra reglas lógicas.

Fuera del modelo: el guardrail es una capa aparte. Se puede aplicar a cualquier texto, sin llamar al modelo, con la API ApplyGuardrail.

CASO DE AGENTES · 13 DE 15

Alucinaciones: cuando suena seguro y es falso

Por qué pasan
El modelo predice lo más probable, no lo verdadero. Sin el dato, rellena el hueco.

En un agente, peor
Un dato inventado en el paso 1 se usa en el 2, en el 3… y llega al informe.

En el prompt

· Permitir decir «no lo sé»
· Citar la fuente de cada afirmación
· Usar solo los documentos dados

En la arquitectura

· RAG con fuentes citadas
· Comprobar el anclaje (guardrail)
· Repetir y comparar respuestas

En el proceso

· Agente de citas que verifica
· Medirlas en las evaluaciones
· Persona en las decisiones serias

Se reducen, no se eliminan. Lo importante se comprueba siempre.

Seguridad: el agente es un usuario más

Inyección de prompt
órdenes ocultas en lo que lee

Permisos excesivos
puede más de lo que necesita

Fuga de datos
datos internos en apps de IA

IA en la sombra
agentes que nadie controla

Cómo lo plantea Zscaler: confianza cero también para los agentes

Descubrir
Inventario de agentes, servidores MCP e IA dentro del SaaS.

AI Broker
En medio del tráfico MCP y entre agentes: qué puede usar cada uno.

AI Access Graph
Mapa de identidades, apps y datos; detecta accesos de más.

Acceso seguro a la IA
Controles sobre los prompts y guardrails por intención.

Red teaming
Atacar los servidores MCP propios antes que nadie.

En el dispositivo
IDEs con IA, modelos locales y extensiones del navegador.

Capas que se suman: Bedrock Guardrails protege lo que entra y sale del modelo; Zscaler, el tráfico y los accesos de toda la empresa.

CASO DE AGENTES · 15 DE 15

Lo que les salió mal a los agentes

Demasiados agentes
Los primeros lanzaban hasta 50 subagentes para preguntas simples.

Órdenes vagas
Con instrucciones cortas, repetían búsquedas y dejaban huecos.

Fuentes dudosas
Preferían páginas optimizadas para buscadores a fuentes fiables.

Errores que se acumulan
Un fallo pequeño puede estropear una tarea larga.

Misma lección que Tesla: más autonomía exige mejor supervisión, no menos.

Pregunta para el grupo: ¿qué tarea de vuestro trabajo repartiríais entre varios agentes, y quién revisaría el resultado?

Autónoma: actúa sola dentro de unos límites

Persona
Fija objetivo y límites.

→

IA en bucle
Percibe, decide y actúa una y otra vez.

→

Persona
Supervisa y audita.

Cuándo se usa

Vehículos
Conducción autónoma en zonas y condiciones definidas.

Almacenes
Robots móviles que llevan mercancía por rutas acotadas.

Infraestructura
Sistemas que ajustan recursos informáticos según la carga.

Control humano: bajo en el momento, alto en el diseño y la auditoría.

Riesgo principal: un fallo escala antes de que alguien lo vea.

CASO TESLA AUTOPILOT · 1 DE 8

¿Es Autopilot una IA autónoma?

No: es asistencia a la conducción. El conductor sigue siendo responsable.

«Las funciones actualmente activadas requieren supervisión activa del conductor y no hacen el vehículo autónomo.»

Tesla, página de soporte de FSD (Supervised) · traducción propia

Qué es

Autopilot y FSD (Supervised): ayudas avanzadas a la conducción.

Por qué se estudia

Es el caso más conocido de la distancia entre asistencia y autonomía.

Qué veremos

Entradas, modelos, nivel, riesgos, accidentes y resultados frente a personas.

CASO TESLA AUTOPILOT · 2 DE 8

Qué entradas tiene

Cámaras de 360°
«Tesla Vision»: la fuente principal.

Mapas de navegación
Sencillos, en dos dimensiones.

Sin lidar
Decisión de la empresa, distinta de otros fabricantes.

Ordenador del coche
Procesa las imágenes con redes neuronales.

Qué sensores se han ido retirando

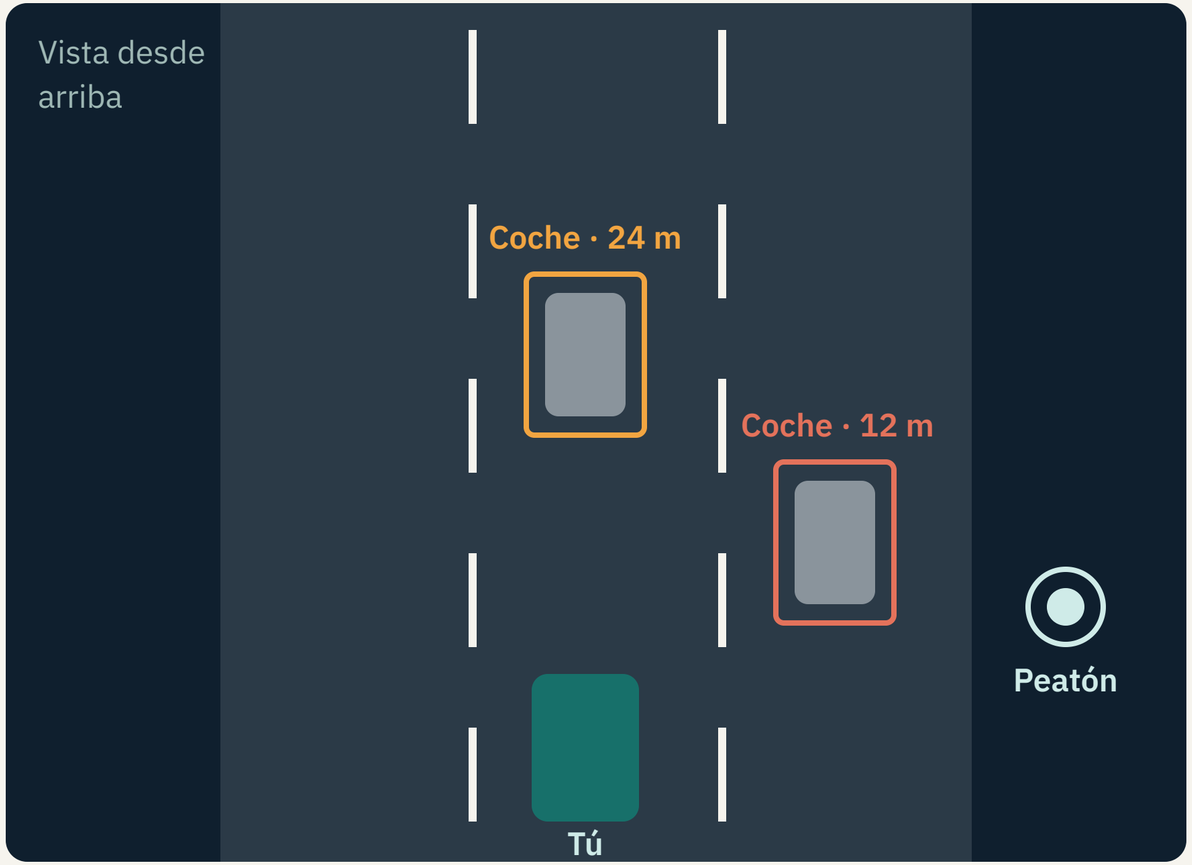
2016: primeros sistemas con radar.

2021: empieza la retirada del radar.

Desde 2022: sin radar ni ultrasonidos en los nuevos.

CASO TESLA AUTOPILOT · QUÉ «ENTIENDE» EL COCHE

De los píxeles a los objetos



1 · Entra
Imágenes: una cuadrícula de números, los píxeles de cada cámara.

2 · La red interpreta
Qué hay, dónde está, a qué distancia y hacia dónde va.

3 · El coche decide
Frenar, mantener el carril o cambiar de velocidad.

Ilustración propia, no una captura real de Tesla.

CASO TESLA AUTOPILOT · 3 DE 8

Cómo procesa con aprendizaje automático

1 · Imágenes
Fotogramas de las cámaras.

→

2 · Red neuronal
Aprende a ver carriles, vehículos y personas.

→

3 · Decisión
Elige cómo moverse.

→

4 · Control
Dirección, acelerador y freno.

Antes

Etapas separadas con reglas escritas a mano.

FSD v12

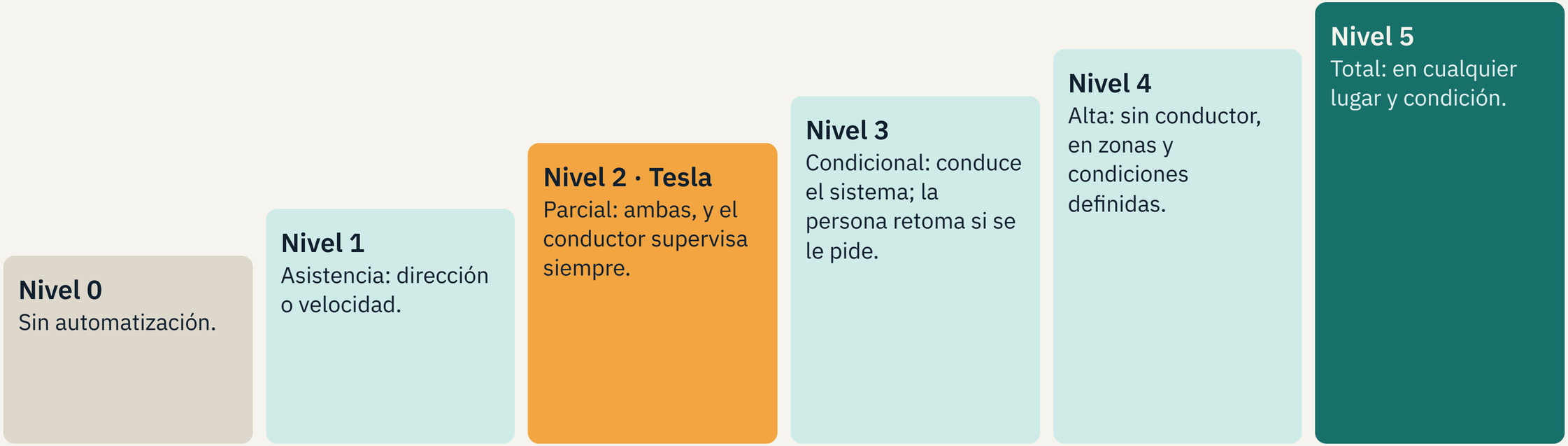
Red de extremo a extremo: de las imágenes a la decisión de conducción.

Con qué aprende

Del comportamiento de millones de conductores, según descripciones públicas.

CASO TESLA AUTOPILOT · 4 DE 8

Nivel en la escala de conducción autónoma



Nivel 2: el conductor debe vigilar siempre y estar listo para tomar el control.

Riesgos del sistema

Riesgo	Qué ocurre	Evidencia
Exceso de confianza	El conductor deja de vigilar	NHTSA habló de una «brecha crítica de seguridad»
Atención mal controlada	El sistema permite un uso poco atento	NHTSA: los controles no aseguraban la atención
Fallos de percepción	No reconoce un obstáculo	2016: remolque blanco contra cielo brillante
Condiciones difíciles	Salidas de la calzada	145 casos con poca adherencia en el análisis de NHTSA
Responsabilidad	No está claro quién responde	Sentencia de Miami, 2025

Idea clave: cuanto mejor funciona una ayuda, más fácil es confiarse.

Accidentes y consecuencias

2016

Williston, Florida

Primer accidente mortal con Autopilot en EE. UU. El coche chocó con un remolque; Tesla dijo que no lo reconoció contra un cielo brillante.

2024

Análisis de NHTSA

956 accidentes revisados; 13 incidentes mortales con 14 muertes. Tesla hizo un retiro (recall) en 2023 de todos los vehículos con Autopilot.

2025

Sentencia de Miami

Accidente de 2019 en Florida. Un jurado halló responsable en parte a Tesla; condena de unos 243 millones de dólares. Tesla apelará.

Matiz: en el caso de 2019 el conductor admitió que iba distraído; el jurado repartió la responsabilidad.

CASO TESLA AUTOPILOT · 7 DE 8

¿Mejora el resultado humano?

Tesla · datos propios, 3.er trimestre de 2025

Millas por accidente

Con Autopilot: 6,36 M

993.000 Sin Autopilot

702.000 Media en EE. UU. (2023)

Críticas: datos propios, solo choques con airbag, uso sobre todo en autopista y perfil de conductor distinto.

Waymo · estudio con Swiss Re

3,8 millones de millas sin conductor, en San Francisco y Phoenix

−76 %
reclamaciones por daños materiales

0
por lesiones, frente a 1,11 por millón de millas

Matiz: zonas y condiciones concretas, y estudio en colaboración con la empresa.

CASO TESLA AUTOPILOT · 8 DE 8

Lo que nos enseña el caso

1

Asistencia no es autonomía: el nivel importa.

2

Una persona que supervisa solo sirve si de verdad está atenta.

3

Para decir que mejora a las personas hacen falta datos comparables.

4

La responsabilidad legal se decide caso a caso.

Pregunta para el grupo: ¿dejaríais que un sistema así actuara sin supervisión en vuestro trabajo? ¿Con qué condiciones?

TESLA FRENTE A WAYMO · 1 DE 4

Waymo: el coche que no necesita conductor

Nivel 4: conduce solo, pero únicamente en las zonas donde está autorizado.

Sin conductor

Funciona sin nadie al volante y sin conductor de seguridad.

Servicio de taxi

Se pide con una app y se paga el viaje, como un taxi.

Ciudades concretas

Phoenix, San Francisco, Los Ángeles, Austin, Atlanta y Miami, entre otras.

TESLA FRENTE A WAYMO · 2 DE 4

Dos formas de abordar la conducción

	Tesla (Autopilot / FSD Supervised)	Waymo
Nivel	2: asistencia	4: sin conductor en zonas definidas
Quién conduce	La persona supervisa siempre	El sistema; no hay conductor
Sensores	Solo cámaras	Cámaras, lidar y radar (13, 4 y 6 en la 6.ª generación)
Dónde funciona	Cualquier carretera, con supervisión	Ciudades concretas
Modelo de negocio	Se vende con el coche	Servicio de taxi con app
Si algo falla	Responde en gran parte el conductor	Responde la empresa

Idea clave: Tesla amplía el territorio y cede la responsabilidad; Waymo acota el territorio y asume la responsabilidad.

TESLA FRENTE A WAYMO · 3 DE 4

Resultados frente a conductores humanos

Waymo · 271,3 M de millas sin conductor (hasta junio de 2026)

Menos incidentes que la referencia humana en las mismas zonas

−82 %	−95 %	−82 %	−93 %
con lesiones	graves o peores	con airbag	peatones heridos

Matiz: datos publicados por la propia empresa, con su método de comparación.

Contraste independiente

Swiss Re (aseguradora), 3,8 M de millas: −76 % de reclamaciones por daños materiales y ninguna por lesiones.

Tesla: datos propios, sin datos independientes comparables.

TESLA FRENTE A WAYMO · 4 DE 4

Qué queda por resolver incluso en el nivel 4

Alcance limitado

Solo funciona en zonas y condiciones donde se ha preparado.

Coste y escala

Más sensores y vehículos propios: crecer ciudad a ciudad es lento.

Situaciones nuevas

Fallos de suministro eléctrico u obras imprevistas pueden dejar coches parados.

Confianza social

Aceptación, empleo de conductores y regulación local.

Pregunta para el grupo: ¿qué preferís, un sistema que funcione en todas partes con vigilancia humana o uno sin vigilancia en pocos sitios?

CURIOSIDADES · WAYMO EN LA CALLE REAL

Cuando la gente pone a prueba a los robotaxis

2023

El truco del cono

En San Francisco, el grupo Safe Street Rebel dejaba un cono de tráfico sobre el capó y el coche vacío se quedaba parado. Waymo lo llamó vandalismo.

2025

El «DDoS» de Waymo

Un usuario pidió 50 robotaxis a una calle sin salida, como un ataque de denegación de servicio, pero con coches.

2025

Pausas por seguridad

Waymo suspendió el servicio en Los Ángeles durante las protestas de junio, y en San Francisco tras un apagón en diciembre.

Idea clave: un sistema que obedece reglas con cautela es fácil de bloquear con trucos sencillos.

CURIOSIDADES · ENGAÑAR A LOS DATOS

99 móviles en un carrito y un atasco que no existe

Berlín, 2020
El artista Simon Weckert paseó 99 móviles en un carrito.

→

Google Maps
Interpretó tantos móviles lentos como un atasco en una calle vacía.

→

Lección
Si un sistema se fía de datos que crea la gente, se le puede engañar.

Google respondió: «Ya sea en coche, en carrito o en camello, nos encanta ver usos creativos de Google Maps.» Otros repitieron el truco con solo 20 móviles.

Pregunta para el grupo: ¿qué otros sistemas de IA se pueden engañar con datos falsos?

SESIÓN 1 · HACIA DÓNDE VA

IA general y singularidad: una hipótesis

Qué plantea

Una IA capaz de mejorarse a sí misma, cada vez más rápido, hasta superar a la inteligencia humana.

Qué se discute

Si es posible, cuándo y cómo. No hay consenso científico ni plazos fiables.

Qué hacemos hoy

Investigar seguridad, exigir supervisión y regular los usos de riesgo actuales.

Estado: teoría y debate. No existe hoy un sistema así, y los riesgos reales que ya tenemos son los de la slide anterior.

SESIÓN 1 · FUNDAMENTOS

Dónde mejora la eficiencia operativa

Salud

Apoyo al diagnóstico por imagen.

Banca

Detección de fraude en pagos.

Industria

Mantenimiento predictivo de máquinas.

Comercio

Recomendación y previsión de demanda.

Administración

Atención ciudadana con asistentes virtuales.

Educación

Itinerarios de aprendizaje adaptados.

Nuevas formas de interacción: voz, chat y búsqueda en lenguaje natural cambian cómo se trabaja con los sistemas.

SESIÓN 1 · FUNDAMENTOS

Técnicas básicas de la IA

Técnica	Idea	Dónde la vemos
Reglas y conocimiento	Expertos escriben reglas si-entonces	Sesión 1
Razonamiento impreciso	Grados de verdad en vez de sí o no	Sesión 1
Lenguaje natural	Entender y generar texto	Sesión 1
Robótica	Percibir y actuar en el mundo físico	Sesión 1
Aprendizaje automático	Aprender patrones de los datos	Sesión 2
Redes neuronales	Aprendizaje con capas de neuronas	Sesión 2

Requisitos de un sistema de resolución de problemas

1

Objetivo

Qué se quiere conseguir.

2

Entrada

Datos y estado de partida.

3

Acciones

Qué puede hacer el sistema.

4

Criterio

Cómo saber si funciona.

Ejemplo, ruta de reparto: objetivo, llegar a todos los clientes; entrada, mapa y pedidos; acciones, elegir el siguiente destino; criterio, menos kilómetros y entregas a tiempo.

SESIÓN 1 · RESOLUCIÓN DE PROBLEMAS

Tres modelos de sistemas de IA

Automatización de tareas

Ejecuta una secuencia fija de pasos sin intervención humana.

Útil cuando: el proceso es repetitivo y estable.

Razonamiento impreciso

Decide con grados de verdad, no solo con sí o no.

Útil cuando: los datos son ambiguos o graduales.

Basados en reglas

Aplica reglas si-entonces escritas por personas expertas.

Útil cuando: el conocimiento se puede explicar.

Cómo elegir: ¿hay reglas claras? ¿hay ambigüedad? ¿hay que poder explicar cada decisión?

SESIÓN 1 · RESOLUCIÓN DE PROBLEMAS

Automatización de tareas

Ejemplo: registrar facturas recibidas por correo (automatización robótica de procesos, RPA).

Llega el correo → **Extrae importe y fecha** → **Registra en contabilidad**

Ventaja: rápida, sin errores de copia y trazable.

Límite: si cambia el formato de la factura, el proceso se rompe.

SESIÓN 1 · RESOLUCIÓN DE PROBLEMAS

Razonamiento impreciso: lógica difusa

En vez de «hace calor: sí o no», cada temperatura tiene un grado de pertenencia entre 0 y 1 a cada categoría.

Temperatura	Frío	Templado	Caliente
18 °C	0,7	0,3	0,0
22 °C	0,0	1,0	0,0
27 °C	0,0	0,4	0,6

Regla difusa: si está caliente, ventilador rápido; si está templado, ventilador lento.

A 27 °C: mezcla ambas reglas y da una velocidad intermedia.

Sistemas basados en reglas

```
def derivar(asunto):
  if "factura" in asunto:
    return "Contabilidad"
  if "contraseña" in asunto:
    return "Soporte"
  return "Revisión manual"
```

Ventajas: explicables, auditables y sin datos de entrenamiento.

Inconvenientes: hay que mantenerlas a mano y no aprenden solas.

SESIÓN 1 · SISTEMAS EXPERTOS

Estructura de un sistema experto

Usuario

Plantea el caso y recibe la respuesta con su explicación.

→

Motor de inferencia

Aplica las reglas a los hechos para llegar a una conclusión.

→

Base de conocimiento

Reglas de la persona experta.

Base de hechos

Datos del caso actual.

Quién la construye: el ingeniero del conocimiento entrevista a la persona experta y traduce su criterio a reglas.

Estrategias de control: encadenamiento

```
R1: si no enciende y cable conectado → fallo de fuente
R2: si fallo de fuente → llamar a técnico
```

Hacia delante

Parte de los hechos

Hechos: no enciende, cable conectado. Se dispara R1 y luego R2: llamar a técnico.

Hacia atrás

Parte de la hipótesis

Hipótesis: llamar a técnico. Pide R2, que exige fallo de fuente, y eso exige comprobar R1.

SESIÓN 1 · LENGUAJE NATURAL

Procesamiento del lenguaje natural

El PLN permite que un sistema lea, interprete y genere lenguaje humano.

Clasificar

Spam, temas, sentimiento de una opinión.

Extraer

Nombres, fechas e importes de un texto.

Transformar

Traducir, resumir y reescribir.

Conversar

Asistentes y chatbots de atención.

Idea clave: el sistema no «entiende» como una persona: convierte el texto en números y busca patrones.

SESIÓN 1 · LENGUAJE NATURAL

Potencial y limitaciones del PLN

Potencial	Limitación
Procesa miles de textos en poco tiempo	Ambigüedad, ironía y contexto
Traduce y resume con buena calidad media	Puede afirmar con seguridad algo falso
Se adapta a muchos dominios	Peor rendimiento en idiomas con pocos datos
Reduce trabajo manual repetitivo	Reproduce sesgos de los textos de partida

Papel del lingüista: define qué significa «correcto», anota ejemplos, evalúa la calidad y trabaja junto a informáticos.

SESIÓN 1 · ROBÓTICA

Sistemas robotizados

Percepción
Sensores y cámaras.

→

Planificación
Qué movimiento hacer.

→

Control
Ordena a los motores.

→

Actuación
El brazo se mueve.

Modelado cinemático: relaciona los ángulos de las articulaciones con la posición de la herramienta.

Programación: enseñando el movimiento al robot o escribiendo el código fuera de línea.

Riesgos legales y éticos

Riesgo	Medida
Privacidad de los datos	Minimizar datos y anonimizar desde el diseño
Errores del sistema	Pruebas, supervisión humana y plan de respuesta
Ataques	Seguridad integrada en el diseño del sistema
Decisiones opacas	Explicar cómo se llega a cada resultado
Responsabilidad	Definir quién responde de cada decisión

Privacy by design: la privacidad se piensa desde el inicio.

Security by design: la seguridad también.

SESIÓN 1 · ÉTICA Y MARCO LEGAL

Sesgos de género en IA y Big Data

De dónde vienen

Datos históricos desbalanceados, etiquetas con prejuicios y equipos poco diversos.

Cómo se detectan

Midiendo el rendimiento del modelo por grupos, no solo la media global.

Cómo se corrigen

Auditar datos, reequilibrar, revisar variables y mantener revisión humana.

Caso conocido: en 2018 la prensa informó de que una herramienta experimental de selección de personal penalizaba currículos asociados a mujeres. [Verificar la fuente antes de citarlo en clase.]

SESIÓN 1 · ÉTICA Y MARCO LEGAL

Marco legal aplicable

Norma	Qué regula
RGPD (UE 2016/679)	Tratamiento de datos personales en la Unión Europea
LOPDGDD (LO 3/2018)	Protección de datos y derechos digitales en España
Reglamento de IA (UE 2024/1689)	Obligaciones según el nivel de riesgo del sistema

Reglamento de IA, idea general: cuanto mayor es el riesgo de un sistema, más obligaciones tiene; algunos usos están prohibidos.

Por verificar: el calendario de aplicación del Reglamento de IA y las normas españolas de desarrollo, antes de cada edición del curso.

SESIÓN 1 · PRÁCTICA EN COLAB

Un mini sistema experto en Python

```
reglas = [
  ({"no_enciende", "cable_ok"}, "fallo_fuente"),
  ({"fallo_fuente"}, "llamar_tecnico"),
]
hechos = {"no_enciende", "cable_ok"}
cambio = True
while cambio:
  cambio = False
  for cond, concl in reglas:
    if cond ⩽ hechos and concl not in hechos:
      hechos.add(concl); cambio = True
print(hechos)
```

1. Ejecuta y comprueba qué hechos nuevos aparecen.

2. Añade una regla y un hecho nuevos.

3. Imprime cada regla que se dispara.

SESIÓN 1 · CIERRE

Ejercicio: ¿qué modelo elegirías?

Caso A

Pasar a contabilidad facturas con siempre el mismo formato.

Caso B

Regular un ventilador según una temperatura que cambia de forma gradual.

Caso C

Justificar a un cliente por qué se le deniega una póliza según criterios escritos.

Opciones: automatización, razonamiento impreciso o sistema basado en reglas. Justifica con los cuatro requisitos del problema.

02

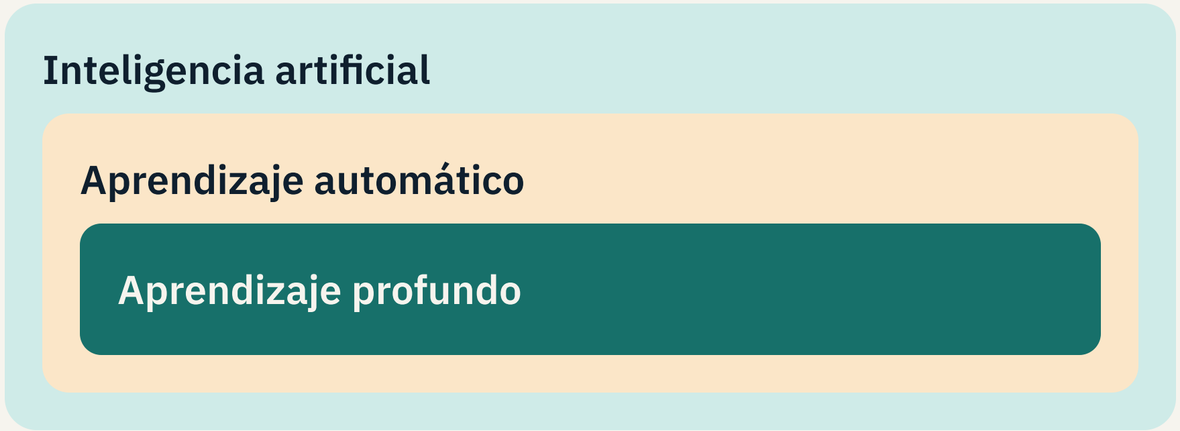
Aprendizaje automático

Modelos que aprenden de los datos: supervisado, no supervisado, refuerzo y redes neuronales.

Sesión 2 · 4 horas · Módulo 5072

SESIÓN 2 · IA, ML Y TIPOS

IA, aprendizaje automático y aprendizaje profundo



IA: cualquier técnica que imita capacidades inteligentes.

ML: aprende patrones de los datos en vez de usar reglas escritas.

Profundo: ML con redes neuronales de muchas capas.

SESIÓN 2 · IA, ML Y TIPOS

Tres tipos de aprendizaje

Supervisado

Aprende de ejemplos con respuesta conocida (etiqueta).

Ejemplo: predecir si un correo es spam.

No supervisado

Busca estructura en datos sin etiqueta.

Ejemplo: agrupar clientes parecidos.

Por refuerzo

Un agente prueba acciones y recibe recompensas.

Ejemplo: un robot que aprende a caminar.

Procedimiento común: datos, identificar patrones, tomar decisiones.

Técnicas y familias de algoritmos

Redes neuronales

Capas de neuronas que ajustan pesos.

Aprendizaje inductivo

De ejemplos concretos a reglas generales.

Razonamiento basado en casos

Resuelve buscando casos parecidos del pasado.

Familias de algoritmos

- Clasificación
- Regresión
- Clustering
- Detección de anomalías
- Refuerzo
- Árboles y reglas

SESIÓN 2 · APRENDIZAJE SUPERVISADO

Datos etiquetados

Superficie (m²)	Habitaciones	Zona	Etiqueta: precio (€)
60	2	Centro	210.000
85	3	Norte	240.000
45	1	Sur	130.000

Variables de entrada: lo que el modelo recibe (superficie, habitaciones, zona).

Etiqueta de salida: la respuesta que debe aprender a predecir.

Fases de un proyecto supervisado

1. Elegir algoritmo
2. Seleccionar datos
3. Construir el modelo
4. Validarlo
5. Ajustar parámetros
6. Probar con datos nuevos
7. Implementar
8. Optimizar

SESIÓN 2 · APRENDIZAJE SUPERVISADO

Entrenamiento, validación y test



Entrenamiento

El modelo aprende de estos datos.

Validación

Sirve para comparar y ajustar parámetros.

Test

Evaluación final, una sola vez.

SESIÓN 2 · APRENDIZAJE SUPERVISADO

Clasificación y regresión

Clasificación

La salida es una categoría.

Ejemplos: spam o no spam; fraude o pago normal.

Regresión

La salida es un número.

Ejemplos: precio de una vivienda; demanda de la semana próxima.

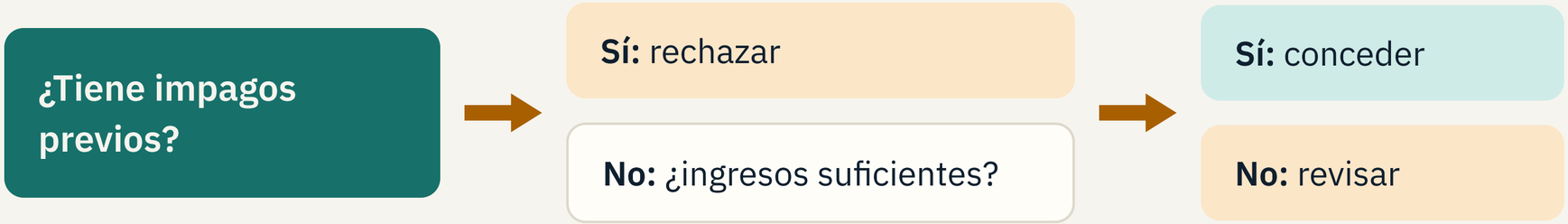
Regresión lineal, la más sencilla:

```
precio = w1 · superficie + w2 · habitaciones + b
```

Aprender significa encontrar los pesos **w** y el sesgo **b** que hacen que las predicciones se parezcan a las etiquetas.

SESIÓN 2 · APRENDIZAJE SUPERVISADO

Árboles y reglas de decisión



Ventaja: cada rama se lee como una regla si-entonces.

Riesgo: un árbol muy profundo memoriza los datos.

Capacidad de generalización

Situación	Error en entrenamiento	Error en test	Qué hacer
Subajuste	Alto	Alto	Modelo más potente o más variables
Buen ajuste	Bajo	Bajo, parecido	Es el objetivo
Sobreajuste	Muy bajo	Alto	Más datos o modelo más simple

Sobreajuste: el modelo memoriza los ejemplos en vez de aprender el patrón, y falla con datos nuevos.

SESIÓN 2 · EVALUACIÓN DEL MODELO

Matriz de confusión

100 correos	Predice spam	Predice no spam
Real: spam	40 · acierto (VP)	10 · se escapa (FN)
Real: no spam	5 · falsa alarma (FP)	45 · acierto (VN)

Falso positivo (FP): un correo bueno marcado como spam.

Falso negativo (FN): un spam que llega a la bandeja.

SESIÓN 2 · EVALUACIÓN DEL MODELO

Métricas a partir de la matriz

Exactitud

`(VP+VN) / total`

85 %

(40+45) / 100

Precisión

`VP / (VP+FP)`

88,9 %

40 / 45

Exhaustividad

`VP / (VP+FN)`

80 %

40 / 50

Cuidado: si el 99 % de los casos es «normal», un modelo que siempre dice «normal» tiene 99 % de exactitud y no sirve.

SESIÓN 2 · NO SUPERVISADO Y REFUERZO

Aprendizaje no supervisado

No hay etiqueta: el algoritmo busca estructura por sí mismo en los datos.

Agrupación

Clientes con hábitos parecidos.

Reducción de dimensión

Resumir muchas variables en pocas.

Anomalías

Detectar lo que no encaja.

Dificultad: sin etiquetas no hay «respuesta correcta»; hay que interpretar y validar los resultados con criterio de negocio.

SESIÓN 2 · NO SUPERVISADO Y REFUERZO

Agrupación con k-means

1

Elegir k y colocar k centros.

2

Asignar cada punto al centro más cercano.

3

Mover cada centro a la media de su grupo.

4

Repetir hasta que los centros no cambien.

Ejemplo: clientes según gasto y frecuencia de compra, en k = 3 grupos.

Límite: hay que decidir k antes y el resultado depende del punto de partida.

Reducción de dimensión

Muchas variables
Por ejemplo, 50 columnas por cliente.

→

PCA
Combina variables en componentes que recogen la mayor variación.

→

Pocas variables
Dos o tres, aptas para dibujar.

Para qué: visualizar, reducir ruido y acelerar otros modelos.

Coste: los componentes son más difíciles de interpretar que las variables originales.

SESIÓN 2 · NO SUPERVISADO Y REFUERZO

Detección de anomalías

Se aprende cómo es lo normal y se marca lo que se aleja de ello.

Usos

Fraude con tarjeta, fallo de un sensor, intrusión en una red.

Cómo

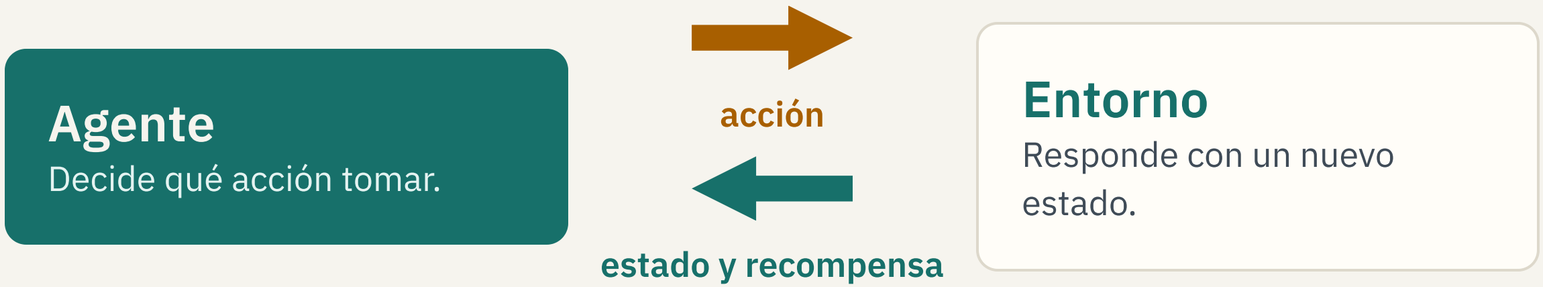
Por distancia al resto, por densidad o por desviación estadística.

Reto

Hay muy pocos casos anómalos y casi nunca están etiquetados.

Conexión con Big Data: con flujos de eventos continuos se detectan anomalías casi en tiempo real.

Aprendizaje por refuerzo



Objetivo: acumular la mayor recompensa a largo plazo.

Ejemplos: juegos, robots, gestión de energía.

SESIÓN 2 · REDES NEURONALES

Cómo aprende una red neuronal

Una neurona:

```
salida = activación(w1·x1 + w2·x2 + b)
```

1

Predecir

Los datos atraviesan las capas.

2

Medir el error

Se compara con la etiqueta real.

3

Propagar hacia atrás

Se calcula cuánto contribuyó cada peso.

4

Ajustar pesos

Y se repite muchas veces.

Redes neuronales convolucionales (CNN)



Filtros: pequeñas ventanas que recorren la imagen y detectan bordes, texturas y formas.

Reducción: resume cada zona para trabajar con menos datos.

Usos habituales: reconocimiento de imágenes, control de calidad visual e imagen médica.

SESIÓN 2 · REDES NEURONALES

Aprendizaje automático frente a profundo

Aspecto	ML clásico	Aprendizaje profundo
Datos	Suficiente con volúmenes moderados	Suele necesitar muchos datos
Variables	Las diseña la persona	La red aprende sus propias representaciones
Tipo de dato	Tablas y registros	Imagen, audio y texto
Explicabilidad	Mayor, por ejemplo en árboles	Menor
Cómputo	Bajo o medio	Alto, con GPU

SESIÓN 2 · HERRAMIENTAS

Herramientas y plataformas del curso

Google Colab

Notebooks en la nube. Con scikit-learn para ML clásico y Keras o PyTorch para redes.

Databricks

Spark con datos grandes y MLlib para entrenar modelos a escala.

Por verificar: qué funciones de aprendizaje automático de Databricks están disponibles en la edición gratuita, antes de proponerlas como práctica.

SESIÓN 2 · PRÁCTICA EN COLAB

Un árbol de decisión con scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
X, y = load_iris(return_X_y=True)
Xe, Xt, ye, yt = train_test_split(X, y, test_size=0.3, random_state=0)
modelo = DecisionTreeClassifier(max_depth=3).fit(Xe, ye)
pred = modelo.predict(Xt)
print(accuracy_score(yt, pred))
print(confusion_matrix(yt, pred))
```

1. Ejecuta y lee la matriz de confusión.

2. Cambia max_depth y compara error en entrenamiento y en test.

3. Prueba KMeans con k = 3 sin usar las etiquetas.

CIERRE DEL BLOQUE

Cinco ideas para llevarse

1. Elige el modelo más sencillo que resuelva el problema.
2. Los datos de calidad pesan más que el algoritmo.
3. Evalúa siempre con datos que el modelo no ha visto.
4. Busca sesgos y explica las decisiones.
5. Cumple la privacidad y el marco legal desde el diseño.

Pregunta para el grupo: ¿qué proceso de vuestro entorno aplicaría estas cinco ideas primero?